



INSTITUTO TECNOLOGICO SUPERIOR DE CALKINI

INGENIERIA EN SISTEMAS COMPUTACIONALES

TALLER DE SISTEMAS OPERATIVOS

- **INGENIERA:** RUTH BETSAIDA MARTÍNEZ DOMÍNGUEZ

ALUMNO:

- TOMMY GAEL CAN MUT_8979

SEMESTRE: 7°/ FECHA: MARTES/03/NOVIEMBRE/2026

PRÁCTICA 5: CONSUMO DE API REST Y PROCESAMIENTO JSON

PRÁCTICA NO. 5 CONSUMO DE API REST Y PROCESAMIENTO JSON

-INTRODUCCIÓN

Esta práctica conecta la aplicación con un servicio REST, procesa respuestas JSON y representa estados de carga, éxito y error. El mismo patrón deberá aplicarse a una fuente de datos pertinente para el proyecto integrador.

-OBJETIVO

Consumir un servicio REST mediante HTTP, convertir JSON a objetos Dart y representar estados de carga, éxito y error.

-LUGAR

Sala de Cómputo / Aula de clases.

-SEMANA DE EJECUCIÓN

Semana 5

- MATERIAL Y EQUIPO

- Proyecto actualizado
- Conexión a Internet
- Paquete http
- Servicio REST público o API autorizada del proyecto
- Git y GitHub

-DESARROLLO DE LA PRÁCTICA

Trabaje sobre una rama nueva y confirme que el proyecto compila antes de introducir cambios. Reproduzca el ejemplo guiado, comprenda cada bloque y después adapte el aprendizaje al proyecto integrador.

Preparación

Crea practica_05_api y agrega el paquete HTTP:

```
flutter pub add http
```

Se utilizará JSONPlaceholder como API pública didáctica. En Android, verifica que la aplicación tenga permiso de Internet si tu configuración lo requiere.

Código Guiado

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;


void main() => runApp(const MaterialApp(
debugShowCheckedModeBanner: false,
home: UsuariosPage(),
));  class Usuario {
final int id;   final
String nombre;
final String correo;
  Usuario({required this.id, required this.nombre, required this.correo});


  factory Usuario.fromJson(Map<String,dynamic> j) =>
Usuario(     id: j['id'], nombre: j['name'], correo:
j['email'],   );
}
class UsuariosPage extends StatefulWidget {
const UsuariosPage({super.key});
  @override State<UsuariosPage> createState() =>
_UsuariosPageState(); }  class _UsuariosPageState extends
State<UsuariosPage> {   late Future<List<Usuario>> futuro;


  @override void initState()
{     super.initState();
futuro = cargar();
  }


  Future<List<Usuario>> cargar() async {
final r = await http.get(
      Uri.parse('https://jsonplaceholder.typicode.com/users'));
if (r.statusCode != 200) {
      throw Exception('HTTP ${r.statusCode}');
    }
    final List datos = jsonDecode(r.body);
    return datos.map((e) =>
Usuario.fromJson(e)).toList();   }
  @override Widget build(BuildContext context) =>
Scaffold(     appBar: AppBar(title: const Text('Usuarios
REST')),     body: FutureBuilder<List<Usuario>>(
future: futuro,       builder: (context, s) {
        if (s.connectionState == ConnectionState.waiting) {
return const Center(child: CircularProgressIndicator());
        }
        if (s.hasError) return Center(child: Text('Error:
${s.error}'));         final usuarios = s.data ?? [];         return
RefreshIndicator(
          onRefresh: () async => setState(() => futuro =
cargar()),           child: ListView.builder(
itemCount: usuarios.length,



            itemBuilder: (_,i) => ListTile(
              leading: CircleAvatar(child:
Text('${usuarios[i].id}')),
title: Text(usuarios[i].nombre),
subtitle: Text(usuarios[i].correo),
            ),
          ),
        );
      },
    ),
```

```
  );
}
```

Análisis De Flujo

La petición HTTP es asíncrona. await espera la respuesta sin bloquear la interfaz; jsonDecode transforma el texto JSON; el factory constructor crea objetos de dominio; FutureBuilder representa visualmente los estados de la operación.

Vinculación con el proyecto integrador

Consumir una API pertinente o, si el backend aún no está disponible, una API de prueba con el mismo contrato previsto; documentar endpoint, método y JSON.

Criterios de aceptación del avance:

- La solicitud valida el código de estado
- El JSON se convierte a objetos Dart
- Se muestran carga error éxito y lista vacía

Entrega en Github

La práctica 5 debe publicarse en la rama practica-05-api-rest. Antes de editar, el equipo debe partir de develop actualizado. No deben subirse .dart_tool, build, credenciales, llaves ni archivos generados.

```
git checkout develop
git pull origin develop
git checkout -b practica-05-api-rest
# desarrollar y comprobar la
práctica dart format . flutter
analyze flutter test git add .
git commit -m "feat: completa practica 05"
git push -u origin practica-05-api-rest
```

Después del push, crear una pull request hacia develop. Incluir en la descripción: propósito, funciones agregadas, pasos para ejecutar, pruebas realizadas, capturas y problemas pendientes.

Evidencia de entrega	Requisito
Código fuente	Proyecto completo y ejecutable en la rama de la práctica
Evidencia de entrega	**Requisito**
README.md	Objetivo, requisitos, instalación, ejecución y participantes
docs/evidencias/practica-05/	Capturas legibles y, cuando proceda, GIF o video breve mediante enlace
Pruebas	Resultado de flutter analyze, flutter test y casos manuales
Enlaces para Moodle o plataforma	URL de pull request y URL del commit final

Condición de entrega: si el enlace no permite acceder a la versión solicitada o el código no corresponde al commit indicado, la práctica queda sin evidencia verificable hasta que se corrija dentro del plazo autorizado.

- EVALUACIÓN Y RESULTADOS

Ejercicio evaluable.

Consume el endpoint /posts de JSONPlaceholder. Muestra título y cuerpo de al menos 20 publicaciones, incorpora indicador de carga, manejo de error, actualización mediante RefreshIndicator y pantalla de detalle al tocar una publicación.

El resultado debe incluir el ejemplo guiado funcionando, el avance del proyecto integrador, las evidencias solicitadas y los enlaces verificables de GitHub. La rúbrica se aplica sobre el commit final declarado por el equipo.

Criterio	Descripción	Puntos
HTTP	Petición y validación de status	25
JSON/modelo	Conversión a objetos Dart	20
Asincronía	Future/async/await correctos	20
UI/estados	Carga, error, lista y detalle	20
Calidad y GitHub	Organización y evidencias. Incluye README, evidencias, pull request y commit final verificables.	15

-REFERENCIAS

Flutter. Fetch data from the internet. https://docs.flutter.dev/cookbook/networking/fetch-data

Dart. JSON support. https://dart.dev/libraries/dart-convert

Flutter. (s. f.). Fetch data from the internet. Recuperado el 18 de agosto de 2026, de https://docs.flutter.dev/cookbook/networking/fetch-data

Dart. (s. f.). Asynchronous programming. Recuperado el 18 de agosto de 2026, de https://dart.dev/language/async

Dart. (s. f.). JSON and serialization. Recuperado el 18 de agosto de 2026, de https://docs.flutter.dev/data-and-backend/serialization/json

-ANEXOS

- Código fuente y archivos de configuración en el repositorio de GitHub.
- Evidencias en docs/evidencias/practica-05/.
- Reporte de práctica elaborado con la estructura del Anexo A.

4. DESARROLLO DE LA PRÁCTICA

4.1 Preparación del entorno de desarrollo

Primero se preparó el entorno de trabajo en Visual Studio Code, utilizando Flutter para crear y ejecutar el proyecto. Se organizó la estructura de archivos necesaria para desarrollar la aplicación y se configuraron las dependencias correspondientes para permitir la comunicación mediante solicitudes HTTP.

Posteriormente, se verificó que el proyecto pudiera ejecutarse correctamente desde el navegador Google Chrome.

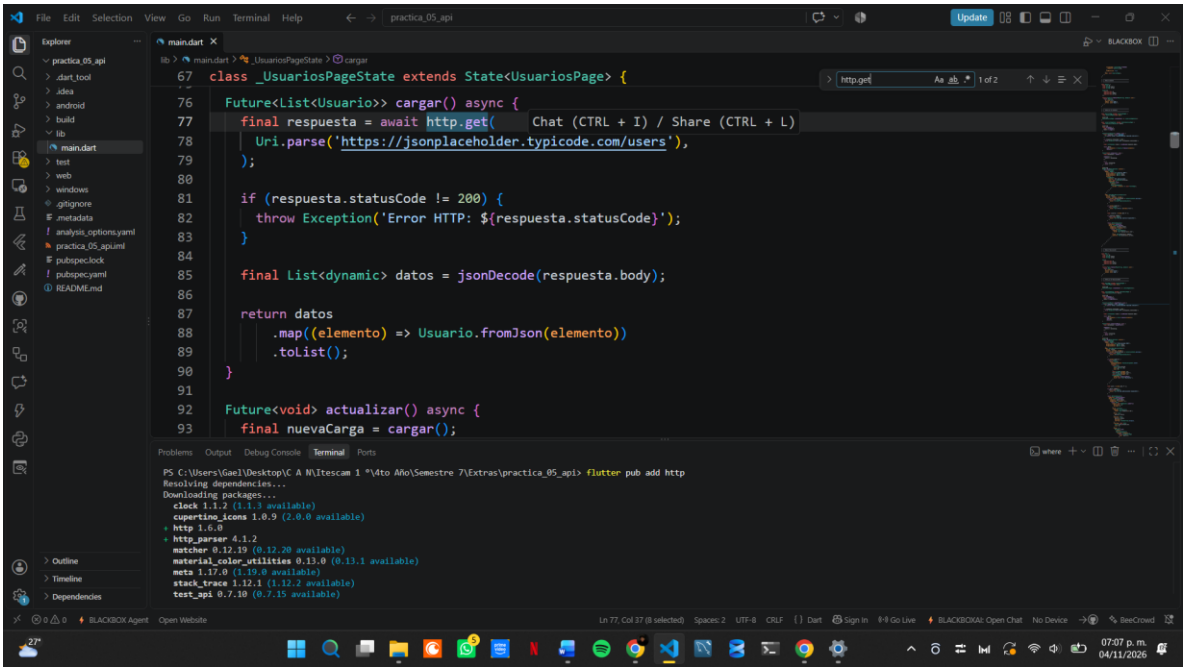
4.2 Conexión con la API REST

Se utilizó JSONPlaceholder como servicio externo para obtener información de prueba. Esta API permitió realizar solicitudes HTTP y recibir respuestas con datos estructurados en formato JSON.

Se trabajó con los siguientes endpoints:

- /users: permite obtener información de los usuarios registrados en el servicio.
- /posts: permite obtener las publicaciones disponibles.

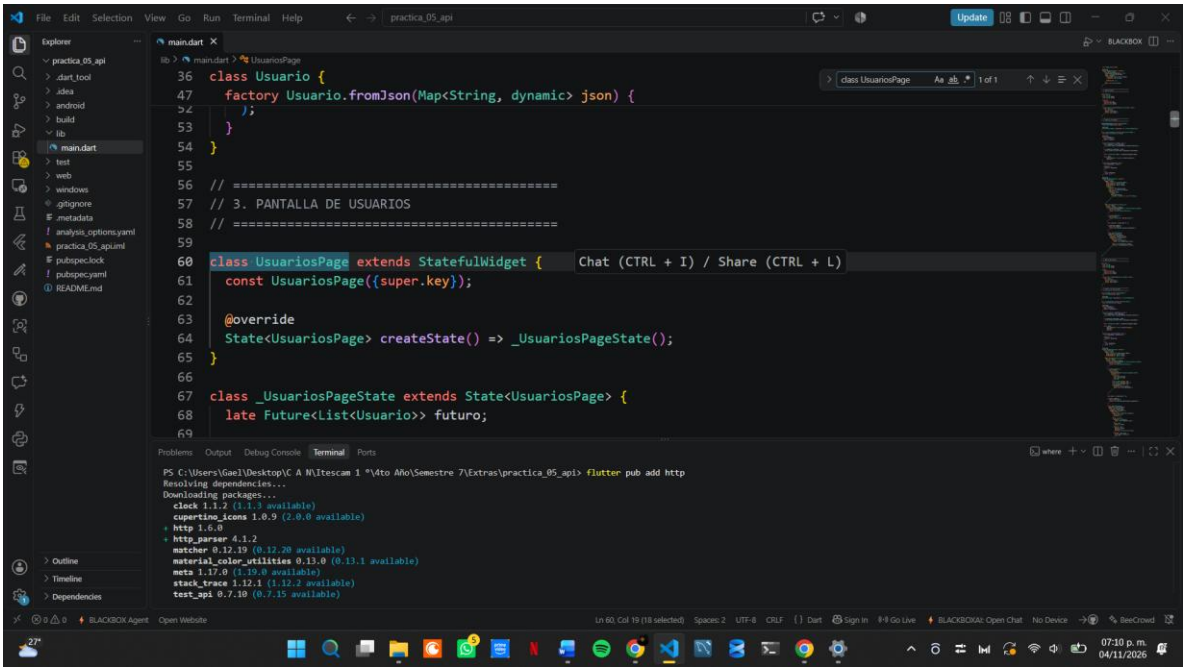
La información recibida fue utilizada para alimentar las diferentes pantallas de la aplicación.



4.3 Obtención y procesamiento de usuarios

Se implementó una pantalla denominada "Usuarios REST", en la cual se muestran los usuarios obtenidos mediante la solicitud realizada a la API.

Cada elemento presenta el identificador del usuario, su nombre y su dirección de correo electrónico. Los datos recibidos fueron procesados para poder mostrarlos de manera organizada dentro de una lista.

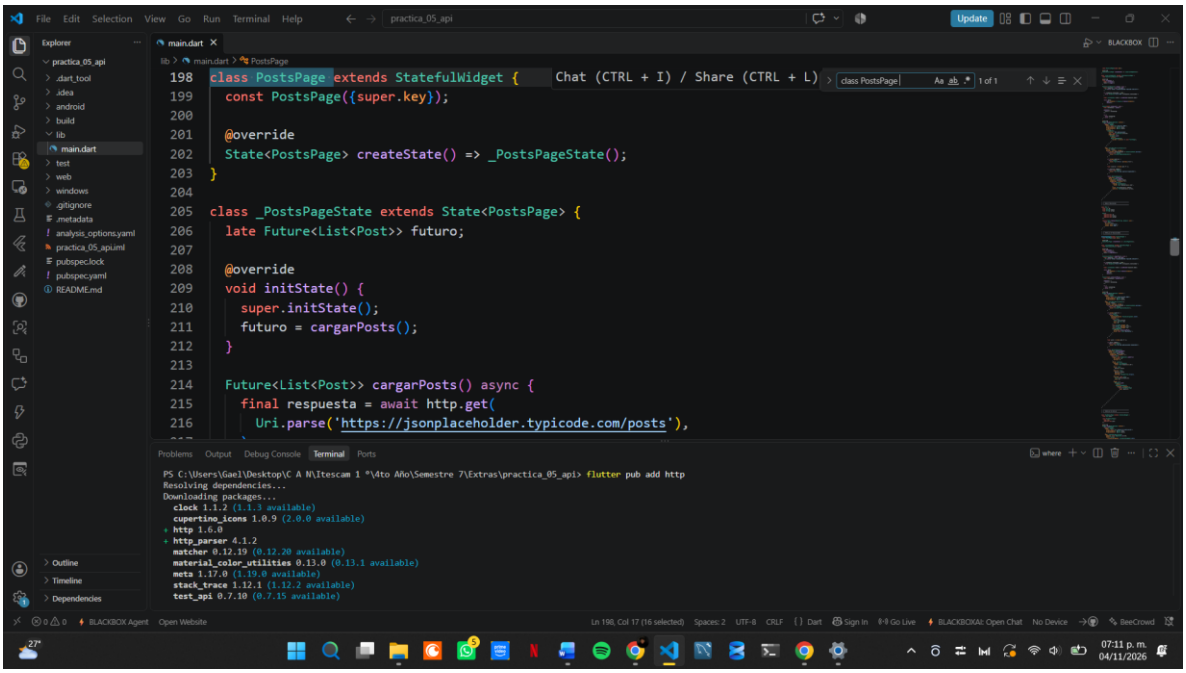


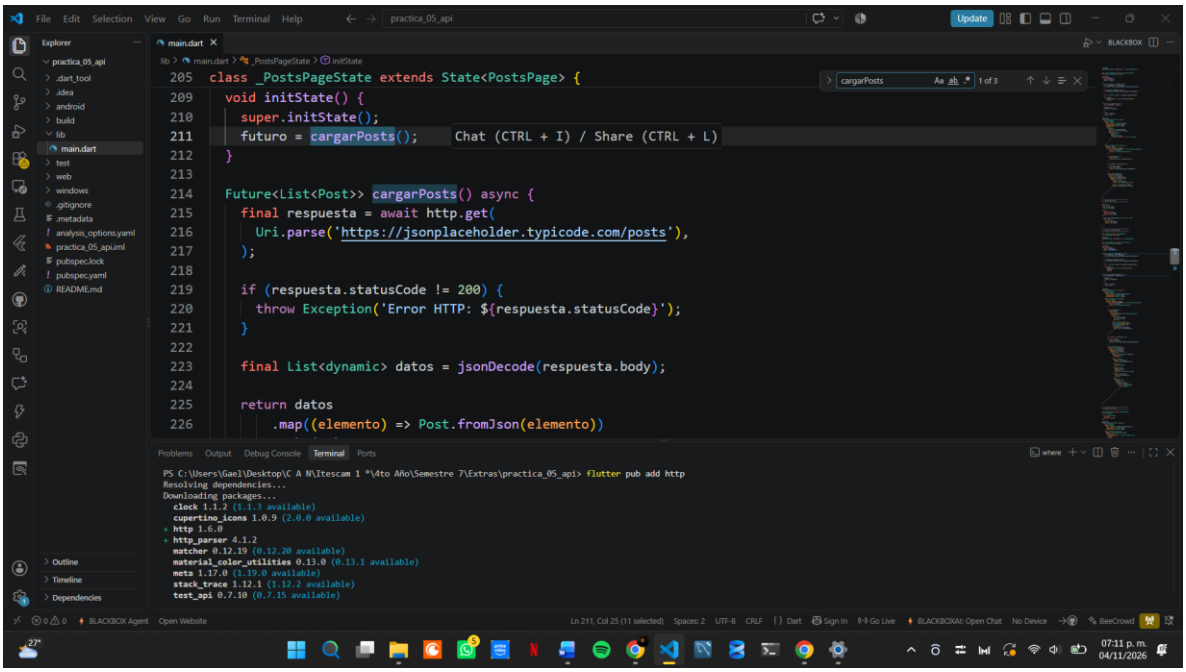
4.4 Visualización de publicaciones

Posteriormente, se desarrolló una segunda pantalla denominada "Publicaciones REST", donde se muestran las publicaciones obtenidas desde el endpoint correspondiente.

Cada publicación presenta su identificador, título y una parte de su contenido. La información se organizó mediante elementos visuales que permiten identificar fácilmente cada registro.

La aplicación permite consultar diferentes publicaciones y desplazarse por la lista para visualizar los datos obtenidos desde el servicio web.


class PostsPage extends StatefulWidget {
const PostsPage({super.key});
@override
State<PostsPage> createState() => _PostsPageState();
}
class _PostsPageState extends State<PostsPage> {
late Future<List<Post>> futuro;
@override
void initState() {
super.initState();
futuro = cargarPosts();
}
Future<List<Post>> cargarPosts() async {
final respuesta = await http.get(
Uri.parse('https://jsonplaceholder.typicode.com/posts'),
PS C:\Users\Gael\Desktop\C A N\Itescam 1 º\4to Año\Semestre 7\Extras\practica_05_api> flutter pub add http
Resolving dependencies...
Downloading packages...
clock 1.1.2 (1.1.3 available)
cupertino_icons 1.0.9 (2.0.0 available)
+ http 1.6.0
+ http_parser 4.1.2
matcher 0.12.19 (0.12.20 available)
material_color_utilities 0.13.0 (0.13.1 available)
meta 1.17.0 (1.19.0 available)
stack_trace 1.12.1 (1.12.2 available)
test_api 0.7.10 (0.7.15 available)

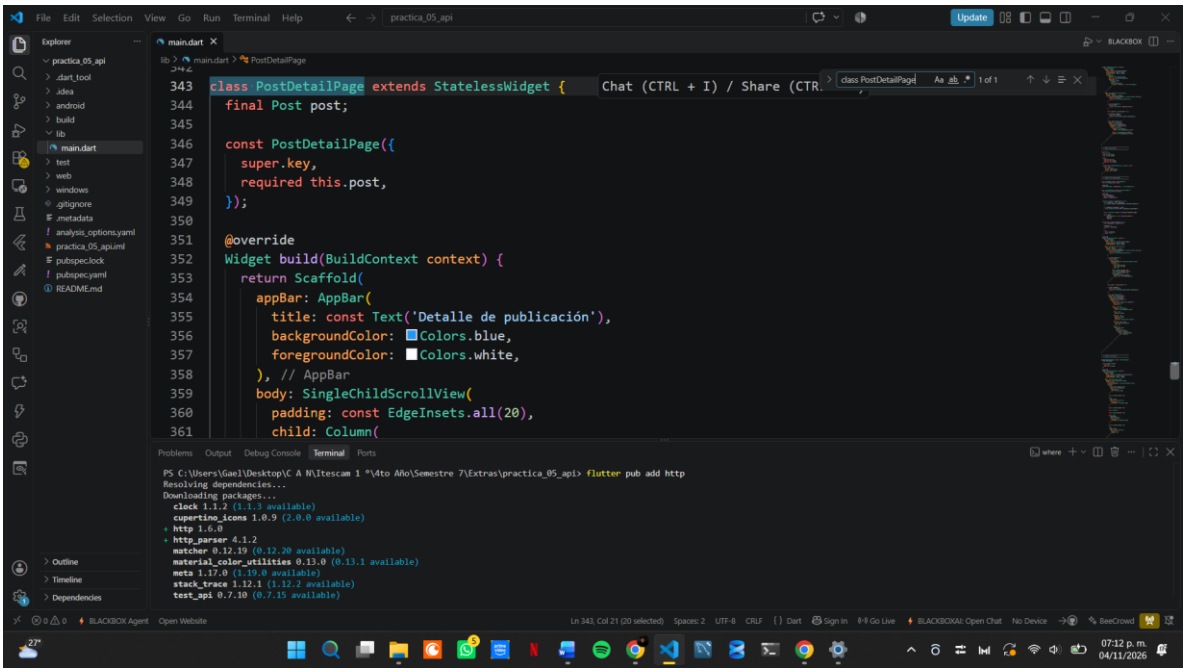

class _PostsPageState extends State<PostsPage> {
void initState() {
super.initState();
futuro = cargarPosts();
}
Future<List<Post>> cargarPosts() async {
final respuesta = await http.get(
Uri.parse('https://jsonplaceholder.typicode.com/posts'),
);
if (respuesta.statusCode != 200) {
throw Exception('Error HTTP: ${respuesta.statusCode}');
}
final List<dynamic> datos = jsonDecode(respuesta.body);
return datos
.map((elemento) => Post.fromJson(elemento))
PS C:\Users\Gael\Desktop\C A N\Itescam 1 º\4to Año\Semestre 7\Extras\practica_05_api> flutter pub add http
Resolving dependencies...
Downloading packages...
clock 1.1.2 (1.1.3 available)
cupertino_icons 1.0.9 (2.0.0 available)
+ http 1.6.0
+ http_parser 4.1.2
matcher 0.12.19 (0.12.20 available)
material_color_utilities 0.13.0 (0.13.1 available)
meta 1.17.0 (1.19.0 available)
stack_trace 1.12.1 (1.12.2 available)
test_api 0.7.10 (0.7.15 available)

4.5 Desarrollo de la pantalla de detalle

Como parte de la práctica, se implementó una pantalla adicional para consultar el contenido completo de una publicación.

Al seleccionar cualquiera de los elementos de la lista, la aplicación navega hacia una nueva vista donde se muestra el número de publicación, su título y el contenido completo.

También se agregó un botón de regreso para facilitar la navegación hacia la pantalla anterior.

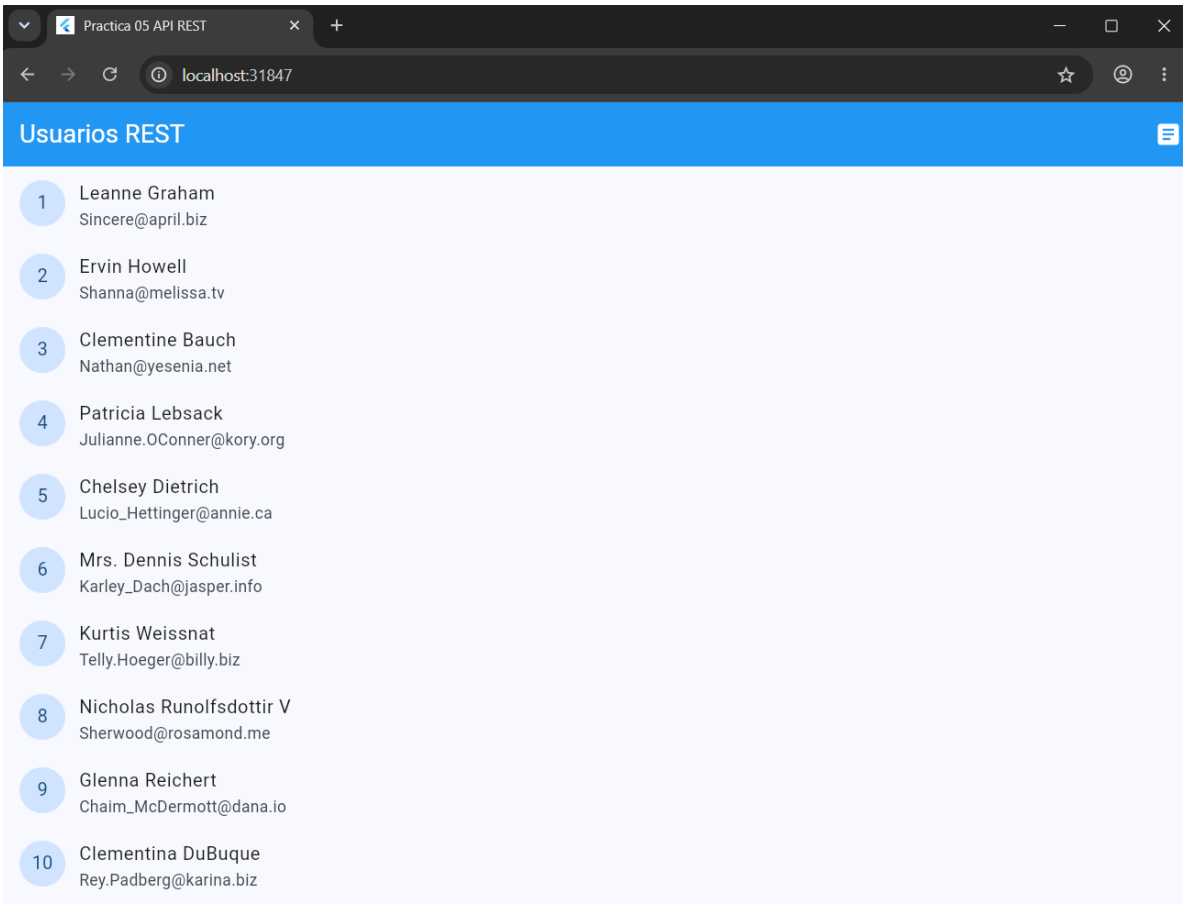


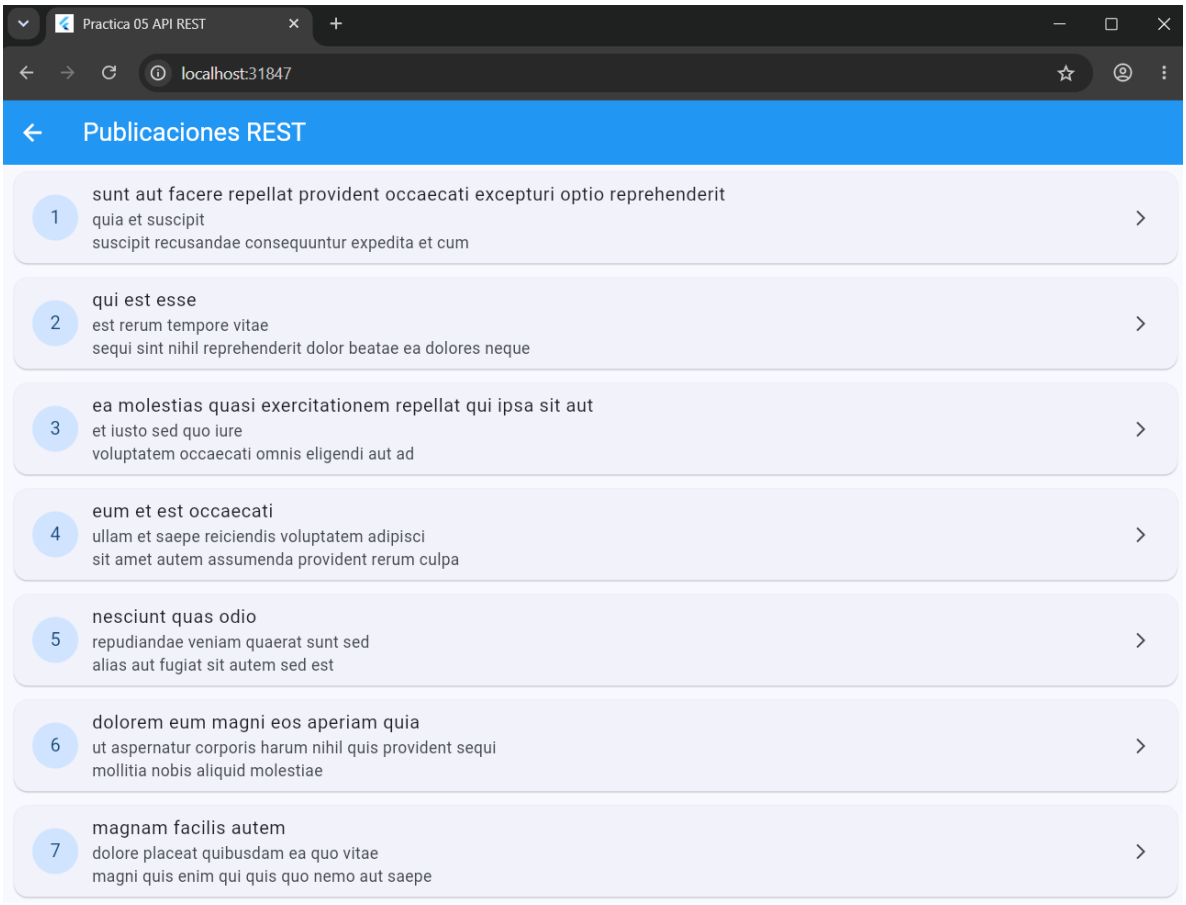
4.6 Ejecución y comprobación

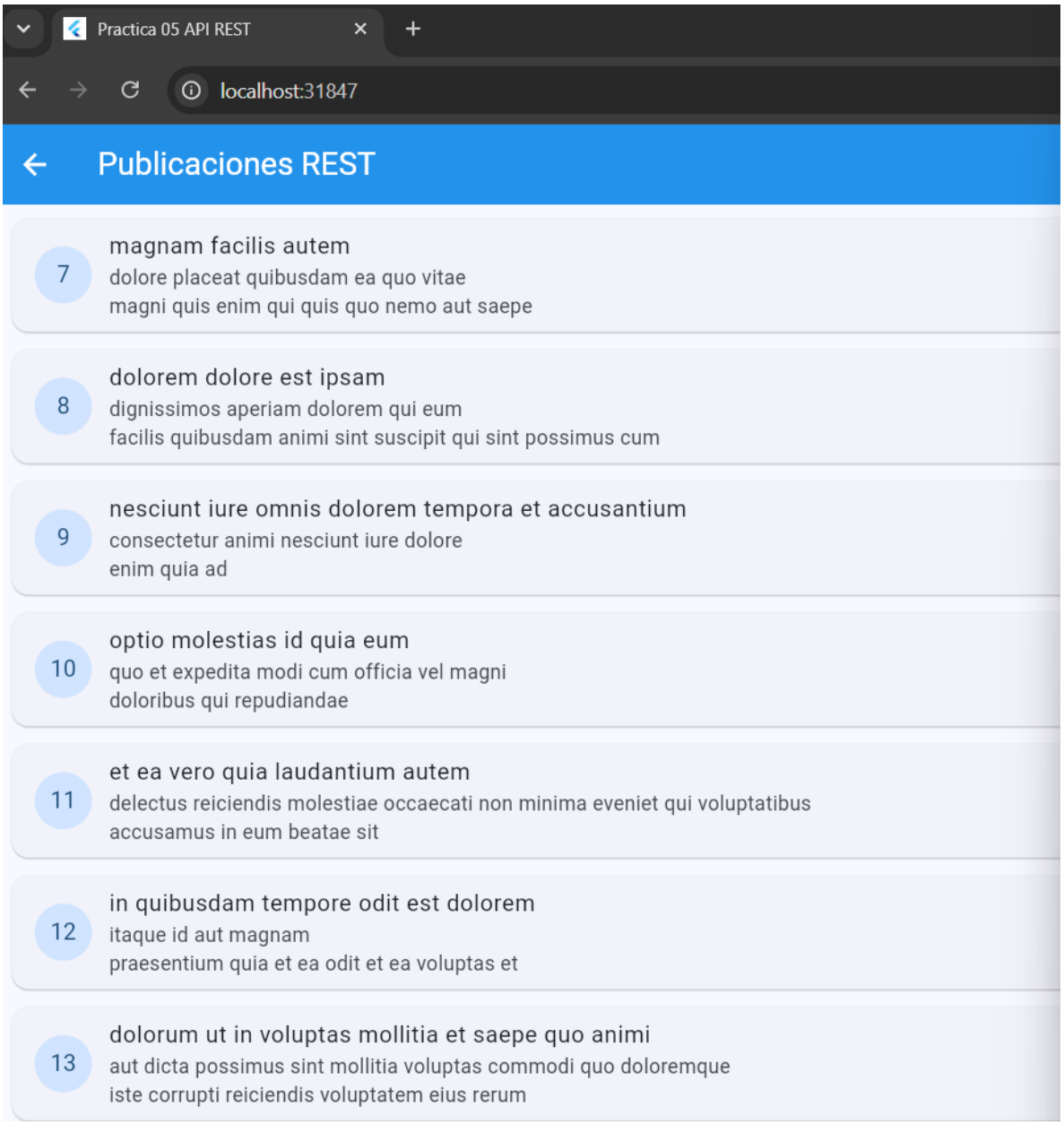
Finalmente, se ejecutó la aplicación utilizando Google Chrome para comprobar el funcionamiento de las diferentes pantallas.

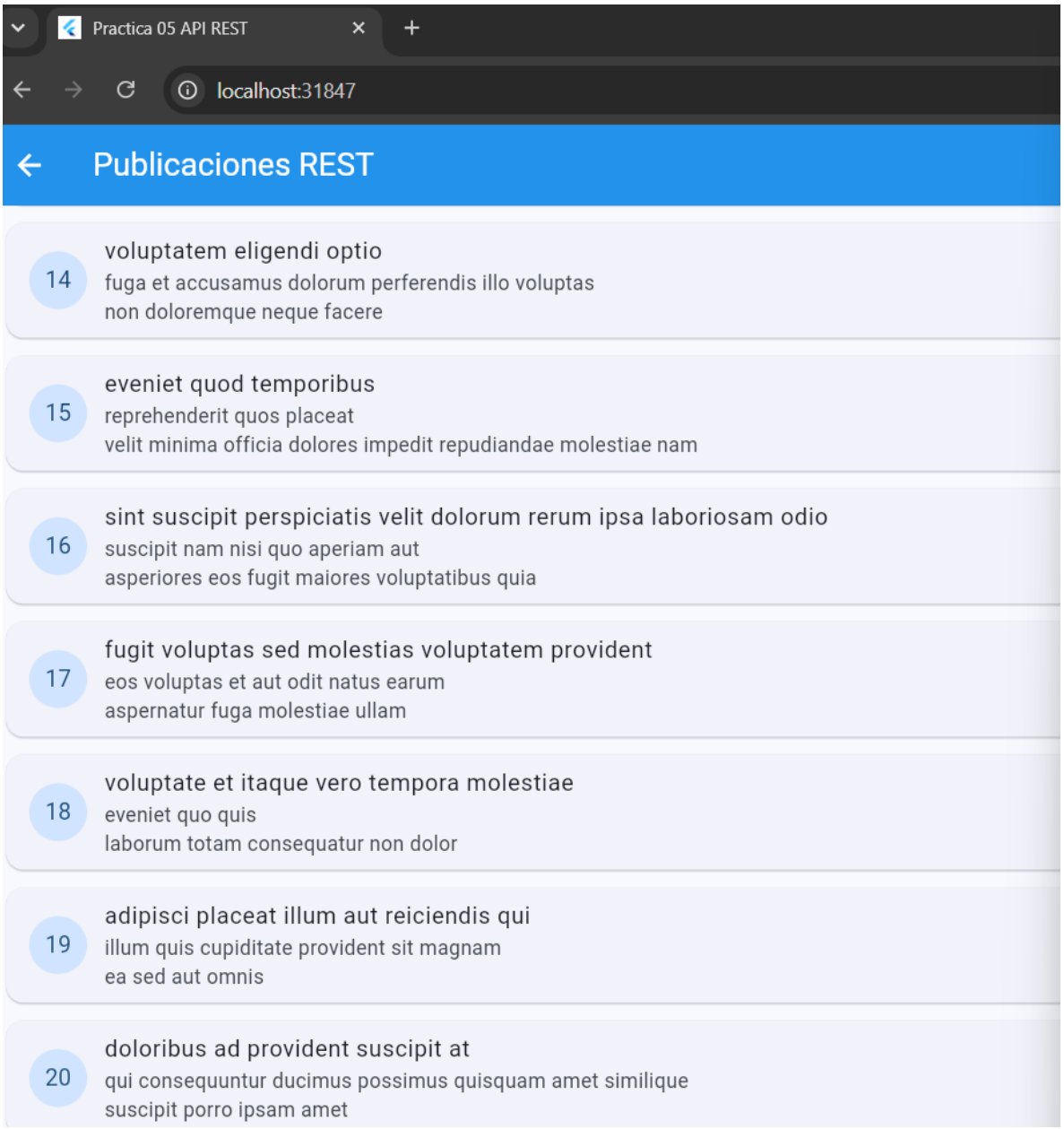
Se verificó que los datos fueran obtenidos correctamente desde la API, que las listas mostraran la información correspondiente y que fuera posible acceder al detalle de cada publicación.

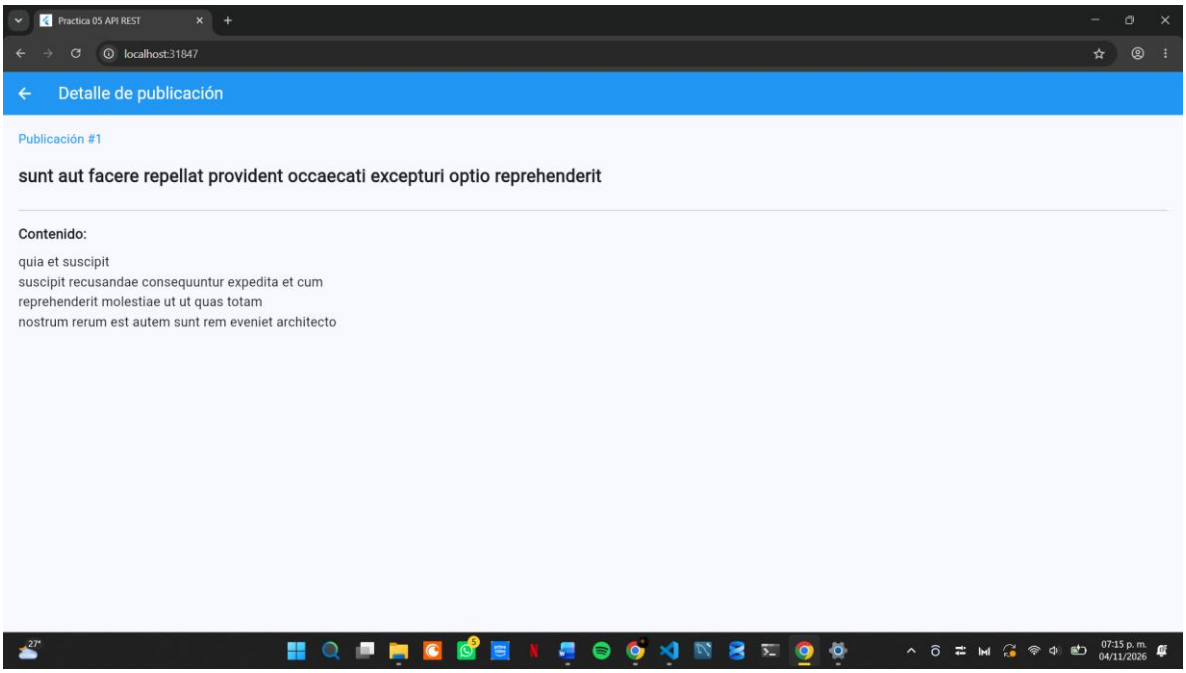
Las pruebas permitieron comprobar la comunicación entre Flutter y el servicio REST, así como la correcta representación de los datos recibidos.

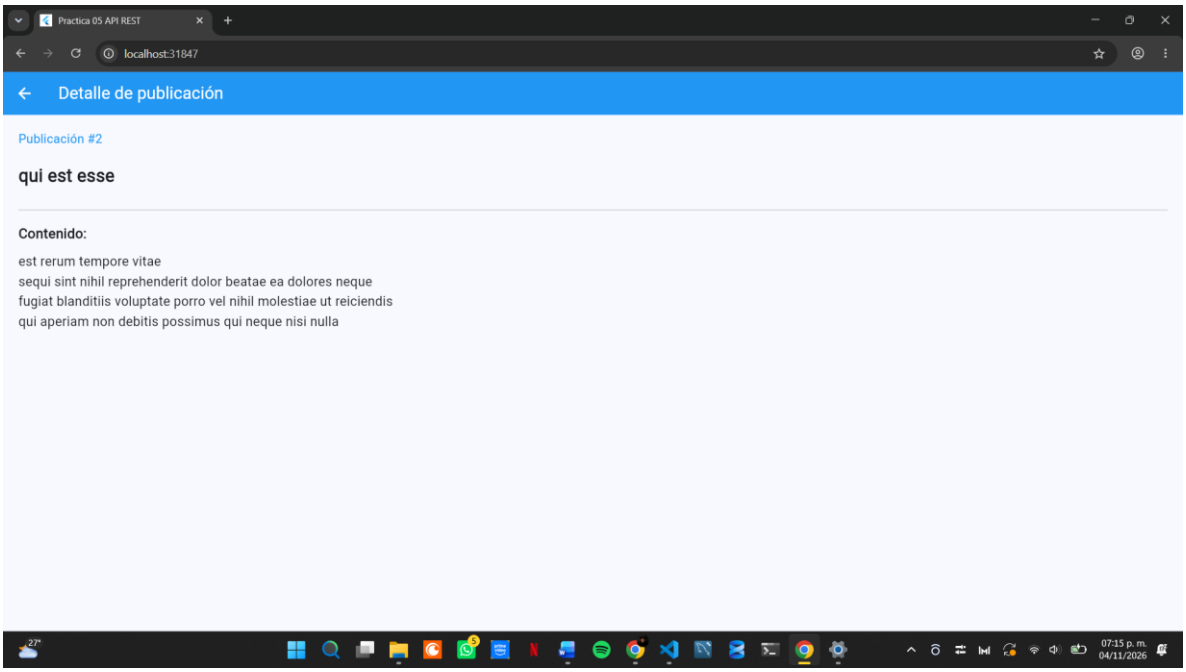

Practica 05 API REST
localhost:31847
Usuarios REST
1
Leanne Graham
Sincere@april.biz
2
Ervin Howell
Shanna@melissa.tv
3
Clementine Bauch
Nathan@yesenia.net
4
Patricia Lebsack
Julianne.OConner@kory.org
5
Chelsey Dietrich
Lucio_Hettinger@annie.ca
6
Mrs. Dennis Schulist
Karley_Dach@jasper.info
7
Kurtis Weissnat
Telly.Hoeger@billy.biz
8
Nicholas Runolfsdottir V
Sherwood@rosamond.me
9
Glenna Reichert
Chaim_McDermott@dana.io
10
Clementina DuBuque
Rey.Padberg@karina.biz


Practica 05 API REST
localhost:31847
Publicaciones REST
1
sunt aut facere repellat provident occaecati excepturi optio reprehenderit
quia et suscipit
suscipit recusandae consequuntur expedita et cum
2
qui est esse
est rerum tempore vitae
sequi sint nihil reprehenderit dolor beatae ea dolores neque
3
ea molestias quasi exercitationem repellat qui ipsa sit aut
et iusto sed quo iure
voluptatem occaecati omnis eligendi aut ad
4
eum et est occaecati
ullam et saepe reiciendis voluptatem adipisci
sit amet autem assumenda provident rerum culpa
5
nesciunt quas odio
repudiandae veniam quaerat sunt sed
alias aut fugiat sit autem sed est
6
dolorem eum magni eos aperiam quia
ut aspernatur corporis harum nihil quis provident sequi
mollitia nobis aliquid molestiae
7
magnam facilis autem
dolore placeat quibusdam ea quo vitae
magni quis enim qui quis quo nemo aut saepe


Practica 05 API REST
localhost:31847
Publicaciones REST
7
magnam facilis autem
dolore placeat quibusdam ea quo vitae
magni quis enim qui quis quo nemo aut saepe
8
dolorem dolore est ipsam
dignissimos aperiam dolorem qui eum
facilis quibusdam animi sint suscipit qui sint possimus cum
9
nesciunt iure omnis dolorem tempora et accusantium
consectetur animi nesciunt iure dolore
enim quia ad
10
optio molestias id quia eum
quo et expedita modi cum officia vel magni
doloribus qui repudiandae
11
et ea vero quia laudantium autem
delectus reiciendis molestiae occaecati non minima eveniet qui voluptatibus
accusamus in eum beatae sit
12
in quibusdam tempore odit est dolorem
itaque id aut magnam
praesentium quia et ea odit et ea voluptas et
13
dolorum ut in voluptas mollitia et saepe quo animi
aut dicta possimus sint mollitia voluptas commodi quo doloremque
iste corrupti reiciendis voluptatem eius rerum


Practica 05 API REST
localhost:31847
Publicaciones REST
14
voluptatem eligendi optio
fuga et accusamus dolorum perferendis illo voluptas
non doloremque neque facere
15
eveniet quod temporibus
reprehenderit quos placeat
velit minima officia dolores impedit repudiandae molestiae nam
16
sint suscipit perspiciatis velit dolorum rerum ipsa laboriosam odio
suscipit nam nisi quo aperiam aut
asperiores eos fugit maiores voluptatibus quia
17
fugit voluptas sed molestias voluptatem provident
eos voluptas et aut odit natus earum
aspernatur fuga molestiae ullam
18
voluptate et itaque vero tempora molestiae
eveniet quo quis
laborum totam consequatur non dolor
19
adipisci placeat illum aut reiciendis qui
illum quis cupiditate provident sit magnam
ea sed aut omnis
20
doloribus ad provident suscipit at
qui consequuntur ducimus possimus quisquam amet similique
suscipit porro ipsam amet


Practica 05 API REST
localhost:31847
Detalle de publicación
Publicación #1
sunt aut facere repellat provident occaecati excepturi optio reprehenderit
Contenido:
quia et suscipit
suscipit recusandae consequuntur expedita et cum
reprehenderit molestiae ut ut quas totam
nostrum rerum est autem sunt rem eveniet architecto


Practica 05 API REST
localhost:31847
Detalle de publicación
Publicación #2
qui est esse
Contenido:
est rerum tempore vitae
sequi sint nihil reprehenderit dolor beatae ea dolores neque
fugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis
qui aperiam non debitis possimus qui neque nisi nulla

5. RESULTADOS OBTENIDOS

Como resultado de la práctica se obtuvo una aplicación funcional capaz de consumir información de un servicio REST externo.

Se logró visualizar la información de los usuarios, consultar una lista de publicaciones y acceder al contenido completo de cada registro mediante una pantalla de detalle.

Además, se comprendió la importancia del procesamiento de respuestas JSON y del uso de solicitudes HTTP para obtener información desde servicios externos.